

Dans ce projet, la méthode **Merise** a été utilisée pour modéliser les données ainsi que leur traitement. Bien que d'autres méthodes comme **UML (Unified Modeling Language)** puissent également être utilisées, Merise offre une approche claire et structurée, particulièrement adaptée aux systèmes d'information.

Trois des six modèles de la méthode Merise ont été mobilisés :

Modèle Conceptuel de Données (MCD)

Ce modèle représente les **entités**, **associations**, **attributs** et **contraintes** du système. Il permet de décrire les besoins métier de manière indépendante de toute technologie.

Modèle Logique de Données (MLD)

Il s'agit de la **traduction du MCD** vers un modèle relationnel, adapté à l'implémentation dans un système de gestion de base de données. On y définit les **clés primaires**, **clés étrangères**, et les **types de données**.

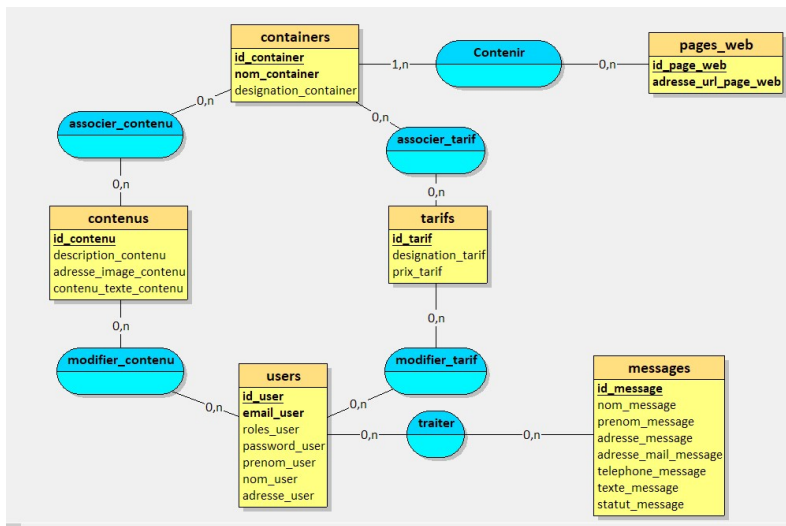
Modèle Physique de Données (MPD)

Ce modèle est le plus technique : il consiste à **traduire le MLD en langage SQL**, avec des considérations de **performance** (index, contraintes, types exacts). Il est **directement exploitable dans un SGBD** comme MySQL, PostgreSQL ou Oracle.

La réalisation a été faite avec le logiciel Looping, qui est un logiciel libre de modélisation conceptuelle de données spécialisé dans la conception de schémas Merise.

Looping

MCD



L'objet du MCD, ou **Modèle Conceptuel de Données**, est de **modéliser de manière abstraite les données et leurs relations** dans un système d'information, **indépendamment de tout logiciel ou base de données**. Chaque rectangle représente une **entité** avec un nom unique et ces attributs (propriété).

Page_web : permet d'identifier chaque page du site ou du contenu sera modifié.

Container : permet d'identifier des zones dans une page qui pourront être modifiées.

Contenu : correspond aux textes ou images qui apparaîtront dans une page.

Tarif : correspond aux informations concernant les tarifs qui apparaîtront dans la page tarif.

User : correspond aux informations concernant les administrateurs du site.

messages : correspond aux informations qui seront transmises par le biais du formulaire de la page contact.

Chaque ovale correspond à une **association** avec un nom et ces cardinalités correspondant aux nombres mini/maxi d'interaction entre les entités. Ici aucun attribut n'a été rajouté mais par exemple si on avait souhaité tracer des modifications, on aurait pu rajouter une date.

Contenir : permet de connaître les containers dans chaque page.

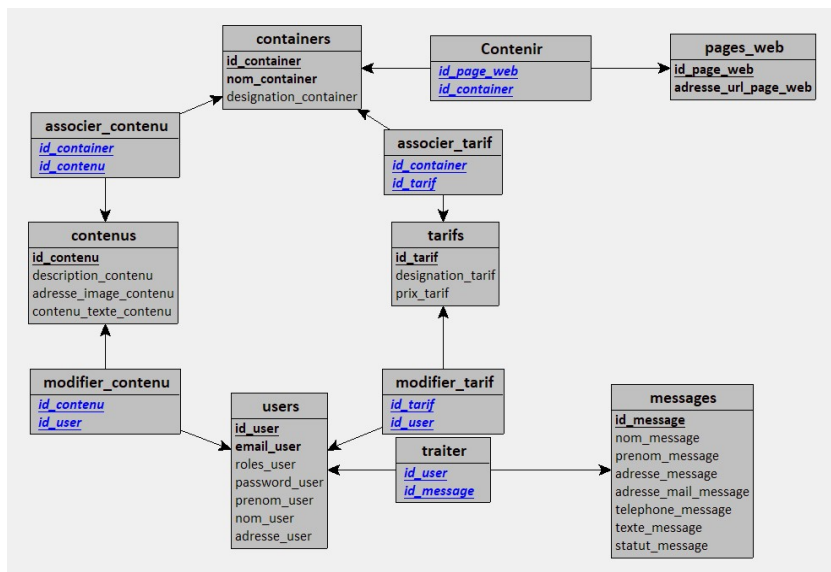
Associer_Contenu : permet de connaître le contenu pour chaque container.

Associer_tarif : permet de connaître le contenu pour chaque container.

Modifier_Contenu et **Modifier_Tarif** : action de l'administrateur pour modifier du contenu ou des tarifs.

Traiter : définit les interventions des administrateurs sur les messages.

MLD



Le MLD (Modèle Logique de Données) traduit le MCD en une structure logique exploitable par un système de gestion de base de données relationnelle, sans encore entrer dans les détails techniques liés à un SGBD (système de gestion de base de données) particulier. Chaque entité et association deviennent des relations (tables) et les attributs, des champs (colonnes). Les clés seront soulignées pour chaque table.

Ici pour les associations de type N:N (many to many), des tables ont été créées avec les deux clés primaires des tables liées.

Si on avait eu des associations de type 1:N (one to many), uniquement une clé étrangère aurait été rajoutée dans la table correspondant à la cardinalité (0 :1 ou 1:1), correspondant à la clé primaire de l'autre table.

Pour une association de type 1:1 (one to one), on aurait pu fusionner les deux tables avec une clé commune. Ou on peut aussi appliquer la règle pour une association de type 1:N.

MPD

Le MPD (Modèle Physique de Données) va traduire le MLD en une implémentation concrète et exécutable dans un Système de Gestion de Base de Données (SGBD) spécifique.

Ici on a un MPD pour un SGBD correspondant à MySQL. On peut voir que pour chaque colonne, le type a été rajouté, définissant le format attendu pour le SGBD choisi.

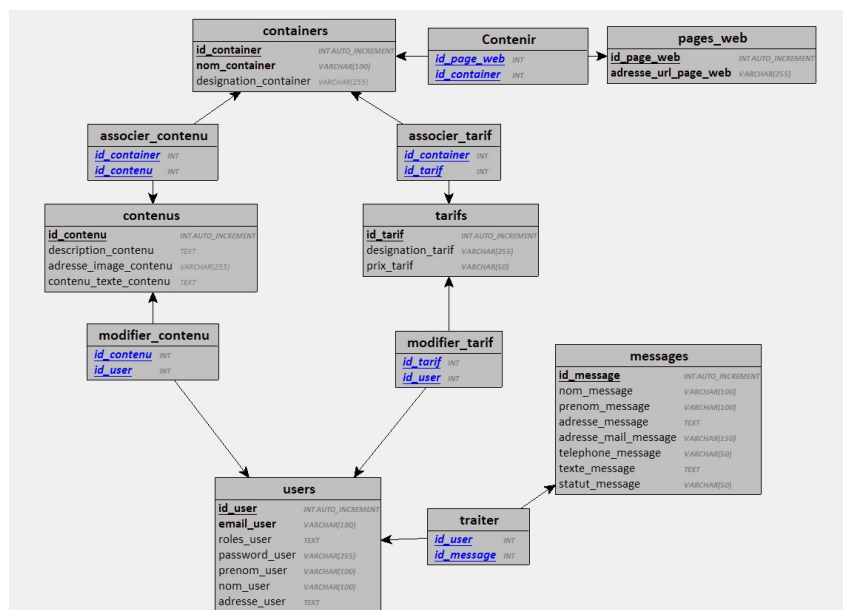
A partir de ce modèle, une requête peut être élaborée pour la création d'une base de données avec toutes les tables et colonnes associées.

Exemple pour containers :

```
CREATE TABLE containers (
  id_container INT AUTO_INCREMENT,
  nom_container VARCHAR(100) NOT NULL,
  designation_container VARCHAR(255),
  PRIMARY KEY(id_container),
  UNIQUE(nom_container)
);
```

Exemple pour contenir :

```
CREATE TABLE contenir (
  id_page_web INT,
  id_container INT,
  PRIMARY KEY(id_page_web, id_container),
  FOREIGN KEY(id_page_web) REFERENCES
  pages_web(id_page_web),
  FOREIGN KEY(id_container) REFERENCES
  containers(id_container)
);
```



CHOIX TECHNOLOGIQUES FRONT END



Pour tout le projet c'est l'éditeur de code Visual Studio Code qui a été utilisé.

Ici pour le front-end, le cahier des charges a spécifié l'utilisation de REACT.

Afin d'assurer une expérience de développement rapide et moderne, j'ai opté pour **Vite** , un outil de build léger et performant. Son principal avantage est sa **vitesse de démarrage** et de **rafraîchissement à chaud** pendant le développement. Pour utiliser Vite avec React, les prérequis suivants sont nécessaires :



- Node Js qui est un environnement d'exécution JavaScript gratuit.



- Npm qui est le gestionnaire de paquets officiel de l'environnement JavaScript, principalement utilisé avec Node.js pour installer les dépendances du projet.

```
PS F:\C-DOSSIER JEROME\INFORMATIQUE\formation developpeur Web\Cours programmation\Exercice\MSPR2> node -v
v24.3.0
11.4.2
```

Lors de l'installation, j'ai choisi la configuration **React avec JavaScript + SWC (Speedy Web Compiler)**, un compilateur moderne plus rapide que Babel, ce qui optimise la compilation du code.

J'ai également intégré plusieurs bibliothèques pour enrichir l'interface :



- Bootstrap, qui est un framework CSS open source pour faciliter la réalisation d'un style responsive.



- REACT Router, une bibliothèque de routage pour React permettant de gérer la navigation entre les différentes pages de l'application via des URL spécifiques.



- Embla Carousel, qui est une bibliothèque légère de carroussel prévu pour REACT. Elle sera utilisée pour le slider et le carroussel de la page d'accueil.

CHOIX TECHNOLOGIQUES BACK END

Ici pour le back-end, le cahier des charges a spécifié l'utilisation de Symfony et de API platform. Symfony est framework PHP basé sur le modèle MVC (Modèle-Vue-Contrôleur), pourvu de composants réutilisables et de nombreuses extensions (bundles). un ensemble Modele Vue Contrôler de composants PHP ainsi qu'un framework MVC libre écrit en PHP. Il fournit des fonctionnalités modulables et adaptables qui permettent de faciliter et d'accélérer le développement d'un site web.

Les prérequis sont :



- Il est nécessaire d'utiliser un logiciel qui permettra de mettre en place un serveur Web local, un interpréteur PHP et une database. J'ai utilisé Xamp qui avait été déjà installé sur mon environnement pendant la formation qui fournit Apache, PHP et MySQL.



- Composer qui est un gestionnaire de dépendances pour PHP.

Optionnel mais pratique pour le développement :

- Symfony CLI, est un outil en ligne de commande officiel fourni par Symfony.

Après création du projet, j'ai installé les bibliothèques PHP suivantes :

- api, avec la commande *composer require api*. Permet d'installer API Platform.

- easyadmin-bundle, avec *composer require easycorp/easyadmin-bundle*. Crée des interfaces d'administration EasyAdmin pour gérer les entités (CRUD).

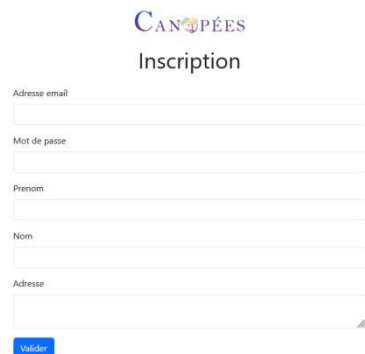
- doctrine-fixtures-bundle, avec *composer require doctrine/doctrine-fixtures-bundle*. Rempli automatiquement la base de données avec des fixtures.

- fakerPHP, avec *composer require --dev xefi/faker-php-symfony*. Génère de fausses données réalistes pour les fixtures.

- Nelmio, avec *composer require nelmio/cors-bundle*. Gère les CORS (Cross-Origin Resource Sharing) pour autoriser le front-end à faire des requêtes HTTP vers le back-end.

ADMINISTRATION DU SITE

En tapant l'adresse du serveur du back-end, on peut accéder à la page d'accueil ou il est possible de se connecter ou de s'inscrire.



La page d'inscription permet d'enregistrer les informations saisie par l'utilisateur dans la table user. Le mot de passe sera hashé pendant ce processus. Le role user est donné par défaut.

```
if ($form->isSubmitted()){
    if ($form->isValid()){
        $user = $form->getData();
        $hashedPassword = $passwordHasher->hashPassword(
            user: $user,
            plainPassword: $user->getPassword()
        );
        $user->setPassword($hashedPassword);
        $entityManager->persist(object: $user);
        $entityManager->flush();
    }
}
```

```
$user->setRoles(roles: ["ROLE_USER"]);
```

La page de connexion permet de se connecter. Si vous avez un rôle administrateur vous pourrez accéder au tableau de bord (/admin), sinon vous un message d'erreur.

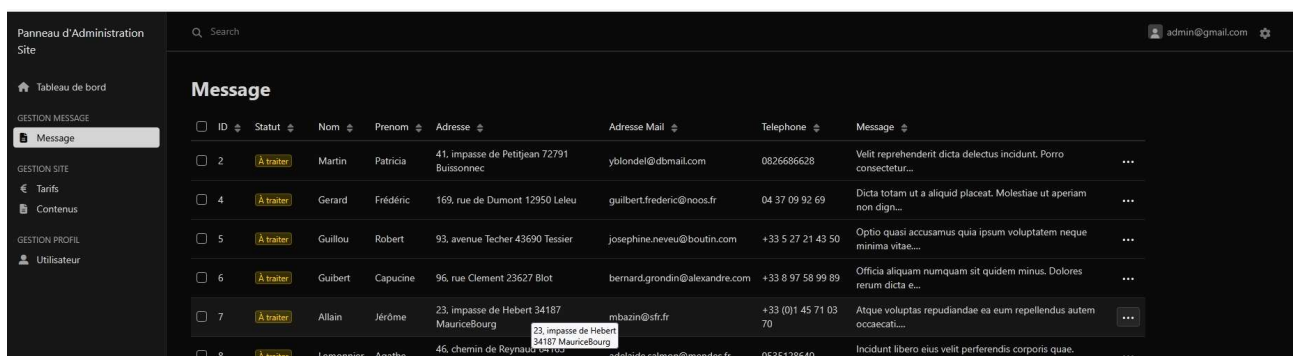
```
<input type="hidden" name="_target_path" value="/admin">

access_control:
    - { path: ^/admin, roles: ROLE_ADMIN }
```



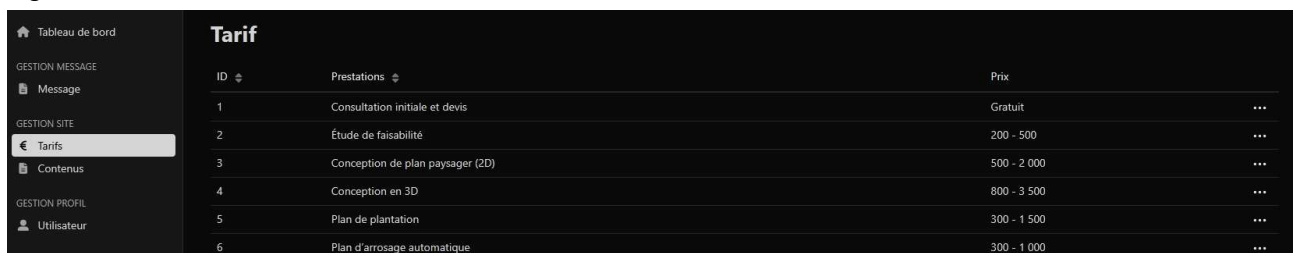
Le tableau de bord donne accès :

- aux Messages afin de les lire, changer de statut, ou les supprimer.
- aux Tarifs et aux contenus afin de pouvoir les modifier.
- aux Utilisateurs ou il sera possible de modifier les informations concernant l'utilisateur connecté.



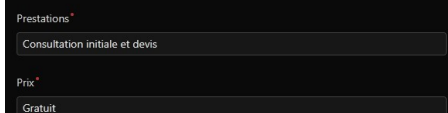
ID	Statut	Nom	Prenom	Adresse	Adresse Mail	Telephone	Message
2	À traiter	Martin	Patricia	41, impasse de Petitjean 72791 Buissonnec	yblondel@dbmail.com	0826686628	Veit reprehenderit dicta delectus incidunt. Porro consectetur...
4	À traiter	Gerard	Frédéric	169, rue de Dumont 12950 Leleu	guilbert.frederic@noos.fr	04 37 09 92 69	Dicta totam ut a aliquid placeat. Molestiae ut aperiam non dign...
5	À traiter	Guillou	Robert	93, avenue Techer 43690 Tessier	josephine.neveu@boutin.com	+33 5 27 21 43 50	Optio quasi accusamus quia ipsum voluptatem neque minima vitae...
6	À traiter	Guibert	Capucine	96, rue Clement 23627 Blot	bernard.grondin@alexandre.com	+33 8 97 58 99 89	Officia aliquam numquam sit quidem minus. Dolores rerum dicta e...
7	À traiter	Allain	Jérôme	23, impasse de Hebert 34187 MauriceBourg	mbazin@sfr.fr	+33 (0)1 45 71 03 70	Atque voluptas repudiandae ea eum repellendus autem occaecati...
8	À traiter	Lemonnier	Anathe	46, chemin de Reynaud 43100	adelajda.salmon@mandes.fr	0535128540	Incidunt libero eius veit perferendis corporis quae...

Page Tarif



ID	Prestations	Prix
1	Consultation initiale et devis	Gratuit
2	Étude de faisabilité	200 - 500
3	Conception de plan paysager (2D)	500 - 2 000
4	Conception en 3D	800 - 3 500
5	Plan de plantation	300 - 1 500
6	Plan d'arrosage automatique	300 - 1 000

Edit Tarif



<https://github.com/JcodeWorld/MSPR2.git>

Page Contenu

Tableau de bord

GESTION MESSAGE

Message

GESTION SITE

Tarifs

Contenus

GESTION PROFIL

Utilisateur

Contenu

ID	Adresse Image Contenu	Description	Texte du Contenu	
1	http://127.0.0.1:8000/uploads/Carrousel/Photo_1_Carrousel.webp	Bois joliment entretenu	Null	...
2	http://127.0.0.1:8000/uploads/Carrousel/Photo_2_Carrousel.webp	Jardin avec un bassin	Null	...
3	http://127.0.0.1:8000/uploads/Carrousel/Photo_3_Carrousel.webp	Magnifique jardin Japonais	Null	...
4	http://127.0.0.1:8000/uploads/Carrousel/Photo_4_Carrousel.webp	Jardin avec des fleurs de toutes les couleurs	Null	...

Edit Contenu

Adresse Image Contenu

http://127.0.0.1:8000/uploads/Carrousel/Photo_1_Carrousel.webp

Description

Bois joliment entretenu

Texte du Contenu

Page Message

Tableau de bord

GESTION MESSAGE

Message

GESTION SITE

Tarifs

Contenus

GESTION PROFIL

Utilisateur

User

ID	Email	Password	Prenom	Nom	Adresse	
11	admin@gmail.com	\$2y\$13\$Lg4eRRISgGo8E4OI/jdDn.M9MtIQb2WVzsKn7sOvFNPsj8GW97zYm	je	bo	adresse	...

1 result

Edit User

Email*

admin@gmail.com

Password*

\$2y\$13\$Lg4eRRISgGo8E4OI/jdDn.M9MtIQb2WVzsKn7sOvFNPsj8GW97zYm

Prenom*

je

Nom*

bo

Adresse*

adresse